# A Persistent Finger-Tree Library for .NET

Design Notes: Architecture, Algorithms, Concurrency, and Testing

Tools.DataStructures.FingerTree
`C:/DataStructures/FingerTree`

Created (UTC): 2026-06-13
Repository HEAD: `c05c26c97c0074a3fb581157b57bfad439e79729`

**Abstract**

This document is a single, navigable tour of `Tools.DataStructures.FingerTree`: a fully persistent (immutable, structurally sharing) data-structure library for .NET 10 built on finger trees. It explains the two engine cores — a purpose-tuned catenable deque and a general monoid-measured tree — the lazy-memoized spine that makes their amortized bounds survive branching persistence in a strict language, the static-abstract "typeclass" encoding of measures and predicates, the allocation-free and closure-free read/split path, the collection and rope families built on top, and the memory-model argument for safe lock-free concurrent reads. It closes with the three-tier testing strategy — example tests, generative property tests, and generative *command-sequence* tests with shrinking — plus the tearable-struct concurrency stress suite and the review evidence that backs the correctness claims. It is written to be read top to bottom by a newcomer and dipped into by a maintainer.

## Contents

---

# 1 Orientation

The library lives in one project, `Tools.DataStructures.FingerTree` (namespace identical), targeting .NET 10 with C# preview features (static abstract interface members in particular). Every public type is **immutable and persistent**: an operation never mutates its receiver; it returns a new version that shares all unchanged structure with the old one. Old versions remain valid forever, which is what makes snapshots, undo histories, and lock-free sharing fall out for free (Section 11).

The design rests on a deliberate split into **two engine cores**, mirroring the Haskell ecosystem's own split between `Data.Sequence` (a tuned size-only sequence) and `Data.FingerTree` (a general measured tree):

- **The tuned deque** — `FingerTreeDeque<T>` — a simplified Claessen-style 2–3 finger tree specialized to the size measure, with hand-optimized digit handling. It is the workhorse sequence: push/pop both ends, index, split, concat, and a sorted-search surface.
- **The general measured tree** — `FingerTree<TElement, TMeasure, TMeasureOps>` — the full Hinze–Paterson construction parameterized by an arbitrary monoidal measure. One monotone-predicate `Split` specializes it into a priority queue, an ordered search tree, an order-statistic tree, an interval index, or a weighted-selection structure, depending only on the measure plugged in.

On top of those cores sit a measure framework (Section 7), a closure-free predicate API (Section 8), a collection family (Section 9), and a rope family (Section 10). Table 2 summarizes the headline complexities; the rest of the document explains how they are achieved and preserved.

## 2 Background: finger trees in one page

A finger tree is a functional sequence representation that gives amortized constant-time access to both ends and logarithmic split/concat/index. The shape is recursive: a tree is either empty, a single element, or a *deep* node holding a *prefix* digit (1–4 elements), a *middle* subtree one level deeper whose elements are 2–3-element *nodes*, and a *suffix* digit (1–4 elements).

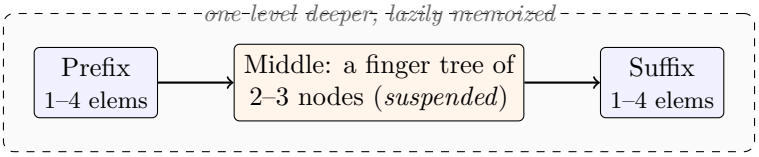


Figure 1: A *deep* finger-tree node. Endpoint access touches only the digits ($\mathcal{O}(1)$); the middle subtree is reached only when an operation descends, and in this library it is a memoized suspension (Section 6).

The amortization argument is the classic one: a push/pop that overflows or underflows a digit pays to repair the spine, but the repaired digits "bank" enough slack that the cost is constant on average. Crucially, that argument assumes the repair is performed *lazily and memoized* — otherwise a program that re-runs an operation on a retained old version pays the worst case every time, and the amortized bound collapses under persistence. Section 6 is about getting that right in a strict language.

For the underlying theory the repository bundles the two canonical sources next to this document: Hinze and Paterson's original paper and Claessen's simplified re-explanation (both as PDFs under `docs/`). This section is only enough to make the design notes self-contained.

## 3 Core I: the tuned deque

`FingerTreeDeque<T>` is the size-measured sequence. Internally it follows Claessen's simplification: nodes carry a cached subtree `Size`, digits are small arrays, and the deep constructor is "smart" (it rebalances overflowing/underflowing digits using the paper's *chop* on pulls). The public surface is large and ergonomic:

- Endpoints: `AddFirst`, `AddLast`, `RemoveFirst`, `RemoveLast`, `First`, `Last`, and `TryPeek`/`TryPop` variants.
- Positional: `this[int]`, `SetItem`, `InsertAt`, `RemoveAt`, `InsertRange`, `RemoveRange`, `GetRange`, `SplitAt`, `SplitItemAt`, `SplitRange`.
- Catenation: `Concat`, `AddRange`.
- Sorted surface (caller maintains order): `SortedLowerBound`, `SortedUpperBound`, `SortedBinarySearch` (deterministic first-equal duplicate semantics), `SortedContains`, `SplitAtSortedLowerBound`/`UpperBound`/`EqualRange`, `InsertSorted`, `RemoveAllSorted`. These exploit cached rightmost-element "signposts" to skip whole subtrees.

It implements `IReadOnlyList<T>` and exposes a struct enumerator. The complexity profile is the finger-tree standard: $\mathcal{O}(1)$ worst-case endpoint reads, $\mathcal{O}(1)$ amortized / $\mathcal{O}(\log n)$ worst-case endpoint updates, $\mathcal{O}(\log(\min(n, m)))$ concat, and $\mathcal{O}(\log(\min(k, n - k)))$ split/index (Table 2).

## 4 Core II: the general measured tree

`FingerTree<TElement, TMeasure, TMeasureOps>` is the Hinze–Paterson construction. Instead of a hard-wired size, every node caches a *measure* drawn from a user-supplied commutative-or-not monoid, and the headline operation is a monotone-predicate split:

```
// Split at the first point where the accumulated measure satisfies the predicate.
(FingerTree<E,M,Ops> Left, FingerTree<E,M,Ops> Right) Split(Func<M,bool> predicate);

// Split around the boundary element, returning it separately.
bool TrySplitFind(Func<M,bool> predicate, out left, out E found, out right);

// Read-only locate: find the boundary element without building result trees.
bool TryLocate(Func<M,bool> predicate, out M measureBefore, out E found);
```

The same code becomes different data structures purely by choice of measure:

| **Measure** | **Structure it induces** |
|---|---|
| size (count) | positional sequence / order-statistic indexing |
| max of a priority | priority queue (`Split` locates the maximum) |
| last-key (rightmost key) | ordered search tree (lower/upper-bound split) |
| size × last-key | order-statistic ordered tree |
| sum (generic-math) | cumulative-weight / weighted-selection structure |
| max interval high-endpoint | interval / stabbing index |

The two cores are kept separate on purpose. The deque can subtract sizes (the size monoid is a group), which enables worst-case-cheaper tricks; a general monoid has no inverse, so the measured tree must take a different route to persistence-robust amortization (Section 6). Fusing them would pessimize the common size-only case, exactly the reason the Haskell ecosystem ships both `Data.Sequence` and `Data.FingerTree`.

## 5 Algorithmic intuition: repair and concatenation

Two operations explain both the amortized bounds and why the spine must be lazy.

**Push repair — the source of amortization.** A push onto a digit that still has room (one to three elements) is $\mathcal{O}(1)$ and touches nothing else. A push onto a *full* four-element digit overflows: the node keeps a two-element digit and pushes a three-element node down into the middle subtree (Figure 2). That descent is the only non-constant work, and it recurs at most $\mathcal{O}(\log n)$ levels. But a digit that just overflowed is now nearly empty, so it banks enough cheap pushes to pay for the next overflow — that is the amortization. The lazy-memoized middle (Section 6) is exactly what preserves this argument when the descent is shared across retained versions.

**Pull repair — the dual on removal.** Removal is the mirror image. Popping an element from a digit that still holds two or more elements is $\mathcal{O}(1)$. Popping the sole element of a one-element digit *underflows*: the node pulls the nearest node up out of the middle subtree and unpacks it into a fresh digit (Figure 3). As with push, the only non-constant work is the descent into the middle, and the freshly refilled digit banks the next several cheap pops — the same amortization, running backwards. When the middle is empty the node instead borrows from the opposite digit, degrading gracefully toward the single/empty base cases.

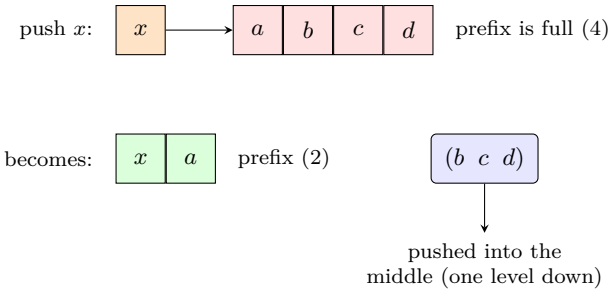


Figure 2: Push repair. Consing $x$ onto a full four-element prefix keeps a two-element prefix $(x, a)$ and pushes the three-element node $(b, c, d)$ down into the middle. Only the (rare) overflow descends; most pushes just extend a non-full digit in $\mathcal{O}(1)$.

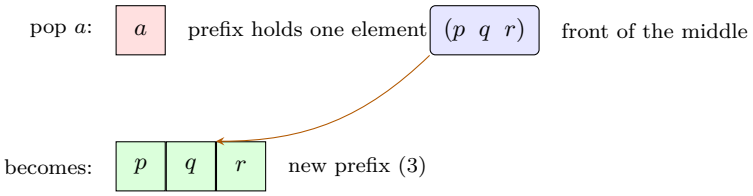


Figure 3: Pull repair, the dual of Figure 2. Popping the sole element of a one-element prefix pulls the front node $(p, q, r)$ up out of the middle and unpacks it into a fresh three-element prefix; only the (rare) underflow descends into the middle.

**Concatenation — merging two spines.** Concatenating two trees keeps the left tree's prefix and the right tree's suffix; the left suffix and the right prefix are regrouped into nodes and folded into a single recursive concatenation of the two middles (Figure 4). The recursion descends both spines in lock-step, so concat is $\mathcal{O}(\log(\min(n, m)))$ rather than linear.

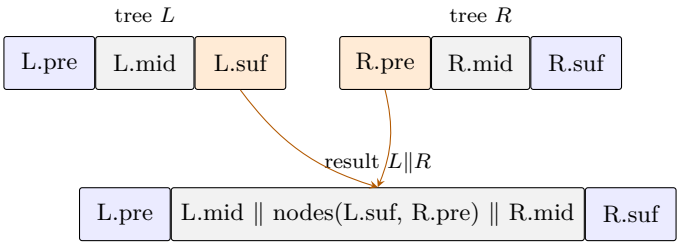


Figure 4: Concatenation. The outer prefix and suffix are inherited from $L$ and $R$; the adjacent inner digits (L.suf and R.pre) are regrouped into nodes and merged into a single recursive concatenation of the middles. Here "‖" denotes sequence concatenation one level down.

## 6 The lazy-memoized spine

This is the load-bearing idea of the whole library, so it gets its own section.

### 6.1 The strict-language problem

The finger-tree amortized bounds depend on the middle subtree being computed lazily and the result being shared. In Haskell that is automatic: thunks are memoized by the runtime. In a strict language like C#, a naive port forces the middle eagerly, so re-running an operation on a *retained* version repeats the full repair — and a program that branches off one old version $k$ times pays $k$ times the worst case. The amortized analysis silently becomes a worst-case-per-branch analysis. Persistence breaks it.

### 6.2 Suspensions, memoized with compare-exchange

The fix follows Hinze and Paterson's strict-language recipe (§3.2 of the paper, as re-explained by Claessen): suspend only the middle subtree of each deep node, and memoize the forced result. The library realizes this with a small explicit cell. For the deque it is `MiddleTree`/`LazyMiddle`/`PendingMiddle`; for the measured tree it is the parallel `MeasuredMiddle`/`LazyMeasuredMiddle`/`PendingMeasured`. The cell holds either the pending operation or, once forced, the computed tree, and publishes the transition atomically:

```
public MeasuredTree<...> Force()
{
    var state = Volatile.Read(ref _state);
    if (state is MeasuredTree<...> computed) return computed;        // already forced
    var result = ((PendingMeasured<...>)state).Run();               // compute (pure)
    var witnessed = Interlocked.CompareExchange(ref _state, result, state);
    return ReferenceEquals(witnessed, state) ? result : (MeasuredTree<...>)witnessed;
}
```

Pending operations are *pure*: running one twice yields structurally identical trees. That is what makes the compare-exchange race benign — two threads may both force, but they compute the same answer and exactly one wins publication; the loser adopts the winner's result. No reader can observe a partially built node. This is also the foundation of the thread-safety claim in Section 11.

### 6.3 Depth-1 suspensions: forcing the source

A subtlety the paper flags: if a push suspension captured *another* suspension as its source, forcing would cascade a chain of arbitrary length. The library follows the paper's discipline — the creator of a push/pop pending operation *forces its source first*, so each suspension is exactly one deferred operation deep. Forcing therefore recurses only down the tree's $\mathcal{O}(\log n)$ depth, never along an unbounded chain, and the amortized analysis is unaffected.

### 6.4 Why the measured tree also memoizes its measure

For the deque, the measure is size, and a popped subtree's size is recovered by subtraction — so the owning node's size is known without forcing. A general monoid has *no inverse*: the measure of a pop suspension cannot be recovered by subtracting the removed node. Consequently each `DeepMeasuredTree` memoizes its own measure lazily, in a boxed cell published by compare-exchange:

```
public override TMeasure Measure
{
    get
    {
        if (Volatile.Read(ref _measureBox) is { } existing) return (TMeasure)existing;
```

```
        var measure = TMonoid.Combine(TMonoid.Combine(CombineAll(Prefix), _middle.Measure
    ),
                                      CombineAll(Suffix));
        var witnessed = Interlocked.CompareExchange(ref _measureBox, (object)measure!,
    null);
        return witnessed is null ? measure : (TMeasure)witnessed;
    }
}
```

A push suspension can still answer its measure arithmetically (the monoidal sum of its forced source and the pushed node) without forcing the structure; only a pop suspension must force. The net consequence, adjudicated in the repository's API-specification defect report, is the *one* place the measured tree differs from the deque: `Measure` is $\mathcal{O}(1)$ *amortized* (not worst-case), while `First`/`Last` stay $\mathcal{O}(1)$ worst-case because they never force the spine. A regression test pins that endpoint reads never force a suspended spine.

## 7 The measure framework

Measures are encoded with C#'s static abstract interface members — a lightweight "typeclass" mechanism that keeps the monoid operations as static, devirtualizable calls on a value-type witness, with no per-element delegate or allocation.

```
public interface IMonoid<TMeasure>
{
    static abstract TMeasure Empty { get; }
    static abstract TMeasure Combine(TMeasure left, TMeasure right);
}

public interface IMeasure<TElement, TMeasure> : ... // Empty, Measure(element), Combine
{
    static abstract TMeasure Measure(TElement element);
}
```

Because `TMeasureOps` is a type parameter constrained to the interface, the JIT monomorphizes each instantiation and the measure calls inline — the abstraction is free at run time.

**Ready-made measures.** `SizeMeasure<T>`, `MaxMeasure`/`MinMeasure`, `KeyMeasure`, and `OrderStatisticMeasure` (with `Optional`/`RankedKey` carriers) cover the common cases. A custom-order strategy `IComparison<T>` (with `DefaultComparison`/`ReverseComparison`) lets the ordered structures take a compile-time comparison witness as well.

**Sum measure.** `SumMeasure<T>` is a generic-math (`INumber<T>`) persistent Fenwick-like structure: `SplitByCumulativeWeight` and `TrySelectByCumulativeWeight` give weighted selection and sampling in $\mathcal{O}(\log n)$ over `int`, `double`, `BigInteger`, and any numeric type.

**Product combinator.** `ProductMeasure<...>` pairs two measures into a `MeasurePair` carrier whose monoid combines component-wise. Named operations (`SplitByFirst`/`SplitBySecond` and the size+sum / size+max / size+min pairings via the `ProductMeasures` factory) infer all the type arguments, so callers write a clean call and get, for example, an order-statistic *and* cumulative-weight structure at once. Product measures nest, so three-component measures compose without new code (Figure 5).

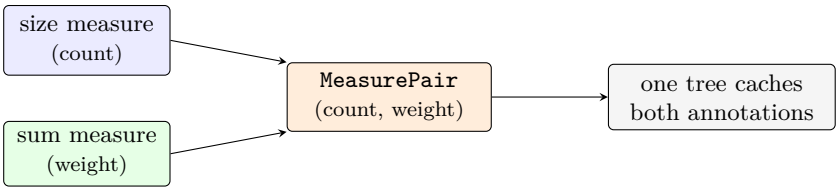


Figure 5: A product measure. Two independent measures combine component-wise into a `MeasurePair` cached at every node, so one structure answers order-statistic *and* weighted-selection queries. Product measures nest, so three or more components compose without new code.

## 8 Closure-free reads and splits

A measure query is a monotone predicate over the accumulated measure. Expressed as a `Func<M,bool>` it allocates a closure on every call — death by a thousand cuts on a hot read path. The library therefore makes the predicate a *value-type strategy*, mirroring the measure typeclass:

```
public interface IMeasurePredicate<TMeasure>
{
    bool Invoke(TMeasure measure);
}

// Generic over the predicate's *type*, so the JIT monomorphizes and inlines it: no
    closure, no boxing.
public bool TryLocate<TPredicate>(TPredicate predicate, out TMeasure before, out TElement
     found)
    where TPredicate : struct, IMeasurePredicate<TMeasure>;
```

The descent itself avoids rebuilding result trees on reads: `LocateTree`/`LocateBuffer` mirror the `SplitTree`/`SplitBuffer` machinery but only walk to the boundary element. Both the read path (`TryLocate`) and the split/mutate path (`Split`, `TrySplitFind`) are generic over the struct predicate; the `Func` overloads simply wrap the delegate in a `FuncMeasurePredicate` adapter for convenience. The collections route their hot operations through the internal struct predicates (`CountAbovePredicate`, `KeyAtLeast`, `PriorityFrontPredicate`, `MaxHighAtLeast`, ...), so they are closure-free throughout.

The payoff is measured, not asserted. `SortedSet.Contains` on $10^5$ elements:

| Implementation path | Time | Allocation |
|---|---|---|
| `Split`-based (builds result trees) | 2,612 ns | 2,400 B |
| `TryLocate` with a `Func` lambda | 351 ns | 96 B |
| `TryLocate` with a struct predicate | ~410 ns | **0 B** |

The predicate interface is public, so callers can write their own zero-allocation queries against a raw tree; the library's own predicates stay internal.

# 9 The collection family

These are thin, ergonomic types built on the general core by choosing a measure and routing reads through struct predicates. All inherit full persistence.

- `SortedBag<T>`, `SortedSet<T>`, `SortedDictionary<TKey,TValue>` — ordered, navigable (floor/ceiling/lower/higher), order-statistic indexable, range-extracting, with full set algebra on the set. Runtime `IComparer` support; the empty instance collapses to a shared singleton only when both empty and default-compared.
- `PriorityQueue<TElement,TPriority>` — meldable, minimum-first, FIFO-stable.
- `IntervalTree<T>` — overlap and point-stabbing queries, overlap counting, value-equal membership/removal with duplicates, and coalescing.
- `ReversibleDeque<T>` — the deque's sibling with $\mathcal{O}(1)$ `Reverse` via a per-node reversal bit and an $\mathcal{O}(1)$ `Mirror`; orientation-aware accessors keep even mixed-orientation `Concat` at $\mathcal{O}(\log \min)$. It is a separate type so its higher constant factor falls only on callers who opt in.

# 10 The rope family

A rope is a persistent sequence chunked for locality: leaves hold contiguous runs rather than single elements, so bulk text-like workloads get cache-friendly storage and $\mathcal{O}(1)$ structural slicing while keeping logarithmic edits.

- `Rope<T>` — element-agnostic positional rope. A `Chunk<T>` is rope-owned `ReadOnlyMemory<T>` plus a cached length; leaves of a `FingerTree<Chunk<T>, int, ...>` measured by total element count. Public imports copy caller memory before publication, while slices share already-owned backing arrays. Index $i$ locates the first chunk whose cumulative length exceeds $i$ (a zero-allocation `TryLocate`), then offsets within it. A chunk-size policy (min 256, max 2048) splits on growth and coalesces on shrink and at concat boundaries to keep chunks healthy.
- `MeasuredRope<T,TMeasure,TMeasureOps>` — the measure-navigable sibling. Each chunk caches a combined user measure; the tree is measured by the product `MeasurePair<int,TMeasure>` (count *and* user measure). Positional operations run on the count component; `Measure`, `PrefixMeasure`, `SplitByMeasure`, and `TryLocateByMeasure` navigate the user measure in $\mathcal{O}(\log n)$ with a bounded within-chunk scan.
- `RopeText` — a companion extension layer that keeps the cores element-agnostic while making text ergonomic: a ready-made `NewlineMeasure`, string interop, $\mathcal{O}(\log n)$ line/column navigation (offset $\leftrightarrow$ (line, column), `GetLine`, `Lines`), and a forward-only `TextReader` adapter.

Editing a $10^6$-element rope is $\sim$2–3 orders of magnitude faster than the equivalent `string` rebuild and flat across size (logarithmic, kilobytes not megabytes); line navigation on a million-line buffer is thousands of times faster than a linear scan. The curated numbers and their honest counterpoints live in `docs/benchmarks.md`.

# 11 Persistence and concurrency

## 11.1 Snapshots and undo are free

Keeping a snapshot is keeping a reference — $\mathcal{O}(1)$, zero allocation, never invalidated because nothing mutates it. An edit allocates only the $\mathcal{O}(\log n)$ of structure that actually changes; the rest

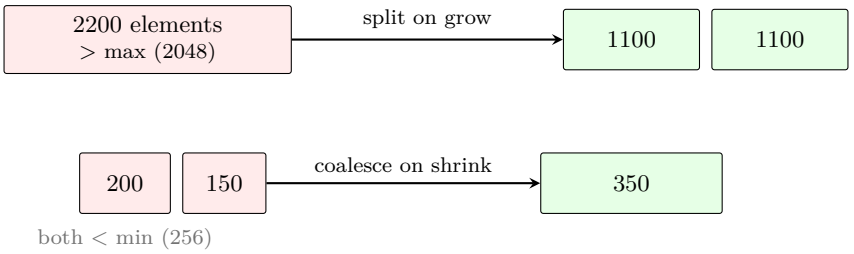


Figure 6: The rope chunk-size policy keeps leaves healthy. An insert that grows a chunk past the maximum splits it (top); adjacent chunks that fall below the minimum coalesce (bottom), including across a concatenation boundary. Healthy chunks bound both the per-edit copy and the tree depth.

is shared. An undo/redo history is therefore a cursor over retained versions, needing no diffing and no inverse operations. A thousand versions cost a thousand small deltas, not a thousand copies.

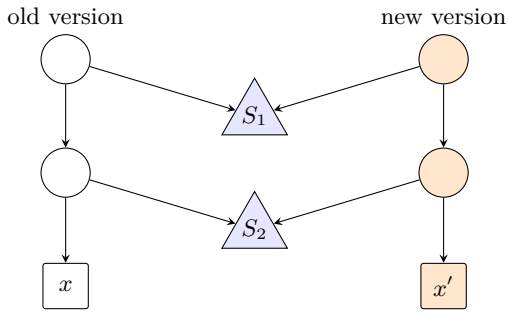


Figure 7: Path copying. Editing leaf $x \to x'$ allocates only the $\mathcal{O}(\log n)$ nodes on the root-to-leaf path (orange); every off-path subtree ($S_1$, $S_2$) is shared by both versions. The old version is untouched and remains valid — this is why a snapshot is free and an edit is logarithmic.

### 11.2 The cost, concretely

The structural-sharing claim is quantitative. Editing a 1,000,000-character rope rebuilds only the $\mathcal{O}(\log n)$ path to the touched chunk plus that one chunk, so a mid-buffer edit allocates single-digit kilobytes regardless of length, while the eager `string` equivalent copies the whole buffer (Table 1). The retained old version is untouched, so a snapshot is a bare reference — zero bytes.

| **Operation on 1,000,000 chars** | **`Rope<char>`** | | **`string`** | |
|---|---|---|---|---|
| insert a short run mid-buffer | 2,939 ns | 5.8 KB | 844,426 ns | 1,953 KB |
| remove a 1,000-char run mid-buffer | 2,217 ns | 7 KB | 507,297 ns | 1,951 KB |
| split in half and re-concatenate | 2,066 ns | 4 KB | 682,576 ns | 1,953 KB |

Table 1: Edit cost on a million-character sequence (BenchmarkDotNet, .NET 10). The rope's per-edit allocation is kilobytes and flat across size; the string's is the whole buffer. Full results and honest counterpoints are in `docs/benchmarks.md`.

The same arithmetic governs history. Keeping $K$ undo states of that buffer costs the one shared structure plus $K$ small deltas — on the order of $K \times 6$ KB, not $K \times 2$ MB. A thousand-step undo

history is therefore a few megabytes of deltas rather than the ∼2 GB a thousand full copies would need. And because the deltas come from the lazy-memoized spine, the per-edit cost stays flat even when every edit branches off the *same* retained version: a prepend on a retained measured tree benchmarks at a flat ∼22 ns from 100 to 1,000,000 elements (Section 6) — the amortized bound holding under branching rather than degrading to an $\mathcal{O}(\log n)$ re-pay per branch. A read does even better: the struct-predicate descent (Section 8) allocates *nothing*, so a query on any version, old or new, costs zero bytes.

### 11.3 Lock-free concurrent reads, and a memory-model argument

Because a version is immutable, any number of threads may read it concurrently with no locks. A producer/consumer setup needs only an atomic publish of a single reference: the producer builds the next version privately and stores it with one volatile write; consumers read that reference and operate on the complete snapshot they captured. No consumer ever sees a half-applied edit.

The subtler claim is that even *first* reads of a freshly built structure are safe, including when the element type `T` or the measure type `TMeasure` is a multi-word struct that the runtime copies non-atomically (a candidate for *torn* reads). The argument, audited in a dedicated review round:

- Element and child storage lives in `readonly` arrays (the deep node's `Prefix`/`Suffix`, the chunk arrays). They are written once at construction and never mutated; the .NET memory model's `readonly`-field freeze plus safe publication of the immutable object graph makes their contents visible before any reader obtains the reference — even on a weak memory model such as ARM.
- Every *mutable* cell holds a *reference* (`object`): the suspension `_state` and the boxed `_measureBox`. They are read with `Volatile.Read` and published with `Interlocked.CompareExchange`, so a large `TMeasure` is boxed and published *atomically* — a reader unboxes a stable box and can never observe a torn field.
- No code path reads an unboxed multi-word `T`/`TMeasure` from a location another thread can concurrently write. Therefore no torn or partially initialized read is possible for any element or measure type.

This is not merely "safe once warmed up": it is safe to share *immediately*, including the lazy work the amortized bounds depend on. The full worked patterns are in `docs/persistence-and-concurrency.md`.

# 12 Worked examples

Three of the ideas above, as code. Each compiles against the public API.

## 12.1 A monotone-predicate split

Every split and locate is driven by a monotone predicate over the *accumulated* measure: the boundary is the first element at which the running measure satisfies the predicate (Figure 8). The descent visits only $\mathcal{O}(\log n)$ nodes — it does not scan the sequence.

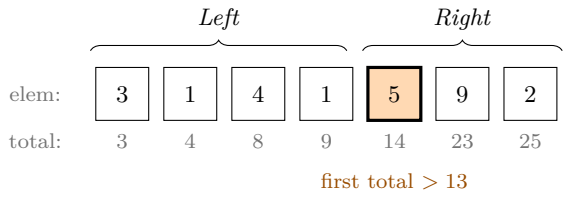


Figure 8: A monotone-predicate split with a sum measure: each cell's running total is shown beneath it. For the predicate "total $> 13$", `Split` returns the prefix *Left* and suffix *Right*; `TrySplitFind` also returns the boundary element (highlighted).

With a sum measure, "cumulative-weight" selection — and therefore weighted random sampling — is a one-liner over exactly this machinery:

```
// Weights 5, 1, 4 (total 10). A threshold in [0,10) selects index 0 with probability
    5/10, etc.
var weights = FingerTree<int, int, SumMeasure<int>>.Create(5, 1, 4);
var draw = random.Next(10);
if (weights.TrySelectByCumulativeWeight(draw, out var picked, out var weightBefore))
{
    // `picked` is the sampled element; `weightBefore` is the summed weight ahead of it.
    O(log n).
}
```

## 12.2 A zero-allocation custom query

Because the predicate is a value-type strategy (Section 8), a caller can supply their own query that runs with no closure and no allocation. Here is an order-statistic lookup — the $k$-th element — over a size-measured tree:

```
readonly struct RankAbove(int rank) : IMeasurePredicate<int>
{
    public bool Invoke(int countSoFar) => countSoFar > rank;   // monotone in the running
     count
}

var tree = FingerTree<int, int, SizeMeasure<int>>.CreateRange(Enumerable.Range(0, 1_000))
    ;
if (tree.TryLocate(new RankAbove(500), out var before, out var element))
{
    // element == the 500th item; before == 500. No result trees built, no delegate
    allocated.
```

```
}
```

### 12.3 Line and column navigation over text

The rope text layer keeps the cores element-agnostic while making editor-style navigation $\mathcal{O}(\log n)$ on the underlying `MeasuredRope<char, int, NewlineMeasure>`:

```
var doc = "first line\nsecond line\nthird".ToTextRope();
int lineCount   = doc.LineCount();                  // 3
var (line, col) = doc.LineColumnOf(offset: 14);     // (1, 3): the 'o' in "second"
int offset      = doc.OffsetOf(line: 2, column: 0);// the start of "third"
string second   = doc.GetLine(1);                   // "second line"
```

### 12.4 Editor-grade text extras

UTF-16 length, Unicode code-point count, and grapheme-cluster count are three different numbers; the text extras expose all three, plus offset conversions, carriage-return-aware lines, and a fluent builder (Section 10):

```
var doc = new RopeBuilder()
    .Append("Hi ").Append(new Rune(0x1F600)).Append(" cafe\u0301\r\nend")   // emoji +
    combining accent + CRLF
    .ToTextRope();

int chars      = doc.Count;             // 16 UTF-16 code units
int codePoints = doc.CodePointCount();  // 15 (the emoji is one code point over two units
    )
int graphemes  = doc.GraphemeCount();   // 13 (base + combining accent is one cluster;
    CRLF is one)

NewlineStyle style = doc.DetectNewlineStyle();          // NewlineStyle.CrLf
string firstLine   = doc.GetLineText(0);                // first line with its trailing
    '\r' removed
int graphemeAt3    = doc.CharOffsetToGraphemeIndex(3);  // 3: character offset 3 is the
    emoji cluster
```

The code-point operations stream over the rope; the grapheme operations materialize the text (cluster boundaries are a global property) but share one segmentation, so counts and offsets always agree.

### 12.5 Running the demos

These ideas run end to end in three small console samples under `samples/`. The *text buffer* tour walks through undo/redo over snapshots, line navigation, and a background reader taking millions of lock-free snapshots while a writer publishes versions; the *measures showcase* exercises the priority queue, weighted sampling, order-statistic set, interval index, reversible deque, and a navigable sorted map — one finger tree under six guises; and the *editor extras* tour shows character / code-point / grapheme addressing, newline-style detection, and the fluent builder. All three are smoke-tested, so they cannot silently drift from the library. Run any with:

```
dotnet run --project samples/Tools.DataStructures.FingerTree.Tour      -c Release
dotnet run --project samples/Tools.DataStructures.FingerTree.Showcase  -c Release
```

```
dotnet run --project samples/Tools.DataStructures.FingerTree.Editor    -c Release
```

## 13 Complexity summary

| Operation | Deque | Measured tree |
|---|---|---|
| First / Last (read) | $\mathcal{O}(1)$ worst-case | $\mathcal{O}(1)$ worst-case |
| Push / pop an end | $\mathcal{O}(1)$ amortized$^\dagger$ | $\mathcal{O}(1)$ amortized$^\dagger$ |
| Measure of whole structure | $\mathcal{O}(1)$ (cached size) | $\mathcal{O}(1)$ amortized$^\dagger$ |
| Index / positional split | $\mathcal{O}(\log(\min(k, n-k)))$ | — (size measure) |
| Predicate split / locate | — | $\mathcal{O}(\log(\min(k, n-k)))$ amortized |
| Concat | $\mathcal{O}(\log(\min(n, m)))$ | $\mathcal{O}(\log(\min(n, m)))$ |
| Reverse (`ReversibleDeque`) | $\mathcal{O}(1)$ | — |

Table 2: Headline complexities. $^\dagger$Amortized bounds hold *under fully persistent (branching) use* because of the lazy-memoized spine (Section 6); worst-case for those rows is $\mathcal{O}(\log n)$.

## 14 Testing strategy

Correctness is established in three complementary tiers plus a concurrency suite, all in one xUnit project (293 tests, warning-free with `CS1591` as an error).

1. **Example and unit tests** — contract tests, edge cases, internal-invariant validation (via `InternalsVisibleTo`) after every step of randomized histories, complexity guards that pin comparer-call counts and zero-allocation endpoint reads, branching-persistence tests, and worked *documentation* examples (snapshots, undo/redo, lock-free reads) that double as runnable proofs for the prose in this document.
2. **Property-based tests (CsCheck)** — thousands of generated inputs with shrinking: rope edit histories versus a `List` oracle with chunk-invariant checks after every step; split/concat and slice laws; text-rope line navigation versus a `string.Split` model with offset round-trips at every offset; the sorted collections versus the framework `SortedSet`/`HashSet` and a multiset model.
3. **Model-based command-sequence tests (CsCheck `SampleModelBased`)** — generate sequences of *operations* (not just data) for the deque, rope, measured tree, measured rope, and sorted set, replay each against a reference model checking equivalence after every step, and shrink a failure to a *minimal failing program* rather than a minimal input. Each immutable structure is wrapped in a mutable cell so an operation can rebind the current version. The per-step check also verifies the full read/split surface at every reached state: the sorted deque's binary search and its three sorted splits, the measured tree's locate/split-find, and the sorted set's navigable queries.

**Concurrency stress.** A dedicated suite uses a deliberately *tearable* 32-byte struct (four longs that must always agree; an `IsIntact` predicate detects any torn read) as *both* element and measure, under shared concurrent reads, concurrent first reads that force the lazy measure boxes, lock-free sliding-window publication, and a long-running branch-and-read soak (duration tunable via the `FINGERTREE_STRESS_SECONDS` environment variable). These empirically corroborate the memory-model argument of Section 11: zero tears across a multi-second soak.

**Adversarial review and mutation evidence.** The concurrency code and the tests were each put through read-only review rounds (a memory-model lens that found the design sound under weak memory, a harness-correctness lens, and a test-soundness lens whose one confirmed finding — a range-removal generator biased to a no-op — was fixed). The model-based harness was further validated by *mutation testing*: deliberately breaking a model operation made the suite fail and shrink to a two-operation counterexample (`[RemoveLast, AddFirst 50]`), and corrupting a split expectation shrank to `[InsertSorted 10]` — direct evidence the tests are not vacuous and that shrinking yields readable minimal programs.

## 15 Repository map

- `src/Tools.DataStructures.FingerTree/` — the library. `FingerTreeDeque` and its `Internal/` core (`Tree`, `Digit`, `Node`, `MiddleTree`, `TreeOperations`); `FingerTree` and `Internal/Measured/`; the measure framework (`Measures`, `BuiltInMeasures`, `SumMeasure`, `ProductMeasure`, `Comparisons`); the predicate API (`MeasurePredicate`, `Internal/MeasurePredicates`); the collections; and the rope family (`Rope`, `MeasuredRope`, `RopeText`).
- `tests/Tools.DataStructures.FingerTree.Tests/` — the three-tier xUnit suite plus the tearable concurrency stress tests.
- `benchmarks/` — the BenchmarkDotNet harness; curated results in `docs/benchmarks.md`.
- `samples/` — three runnable console tours (the text buffer, the measures showcase, and the editor extras of Section 12), each smoke-tested from the suite.
- `docs/` — this document, the normative API specification and its defect report, the persistence/concurrency guide, and the benchmark write-up; the bundled external source papers and reference code are segregated under `docs/external/`.

## 16 References


1. R. Hinze and R. Paterson. *Finger trees: a simple general-purpose data structure.* Journal of Functional Programming, 2006. (Bundled: `docs/external/Finger trees - a simple general-purpose data structure.pdf`.)
2. K. Claessen. *Finger Trees Explained Anew, and Slightly Simplified (Functional Pearl).* Haskell Symposium, 2020. (Bundled: `docs/external/Finger Trees Explained Anew, and Slightly Simplified.pdf`.)
3. Repository documents: `docs/api-specification.md`, `docs/api-specification-defect-report-complexity-guarantees.md`, `docs/persistence-and-concurrency.md`, `docs/benchmarks.md`.